

Language Modeling with Transformers

Kaniz Fatima(267475)

University of Trento

kaniz.fatima@studenti.unitn.it

1. Introduction

This report covers the language modelling exercise on the Penn Treebank corpus, where the task is next-token prediction and quality is measured by perplexity: the exponentiated mean negative log-likelihood of held-out text, so lower is better. Two settings are compared. Part 1.A trains a compact GPT2-style decoder from scratch, improving it one change at a time after first fixing the learning rate. Part 1.B freezes pretrained GPT2 and trains only a small set of low-rank adapters. Together they isolate what pretraining is worth on a corpus of this size. Dataset statistics are given in Table 1.

2. Implementation details

The supplied split gives 42,068 training, 3,370 development and 3,761 test sentences, each ending in an explicit `<eos>` marker. Both parts use the GPT2 BPE tokenizer (vocabulary 50,257); GPT2 has no padding token, so its EOS is reused and masked out of the loss. Part 1.A pads sentences to the longest in each batch, truncated at 128 tokens; Part 1.B packs the corpus into contiguous 64-token blocks, removing padding entirely and yielding 19,729 / 1,561 / 1,768 blocks.

Cross-entropy is accumulated token-weighted over non-padding labels rather than averaged per batch, so perplexity is the true per-token mean. Training uses AdamW with weight decay 0.01 [1] and gradient clipping at 1.0, five epochs maximum with early stopping on development perplexity; the best epoch is the one evaluated on test. A fixed seed precedes every model build, so any configuration reproduces exactly.

On top of the decoder backbone, Part 1.A adds dropout at four points and an optional weight-tying switch that shares the token embedding with the output projection [2]. Part 1.B implements LoRA [3] by hand, without PEFT. GPT2 fuses the query, key and value projections into one `Conv1D`, so that layer is wrapped: the pretrained weights stay frozen while three independent low-rank adapters produce a delta concatenated back into the same $[q \text{ --- } k \text{ --- } v]$ layout, adding 55296r parameters over the twelve blocks. With B zero-initialised the adapter is provably inert at the start – the wrapped model scores 137.09 development perplexity, identical to unmodified GPT2 – which verifies the wiring before any training happens.

Declaration of AI tool use. AI tools were used for code scaffolding and report organisation. All experiments were executed by the author, and every number reported here is the measured output of those runs.

3. Results

Models are selected on development perplexity; the test split is used once, at the end. The learning-rate search (Table 2, Figure 1) selects 10^{-3} at 44.91 development perplexity, with a monotone ranking; 10^{-4} trails badly at 80.66, too slow to converge within the epoch budget.

The incremental search (Table 3, Figure 3) improves

steadily: widening d_{model} and adding depth both help, doubling the attention heads is essentially neutral (43.25 to 43.16), and widening the feed-forward dimension gains on development but not on test. Dropout is the largest single improvement, at 40.49 development and 36.44 test perplexity, and is the selected configuration. All runs satisfy the $PPL < 250$ requirement, the worst being 80.66.

Weight tying, applied last, is the one step that clearly hurts (53.68). The comparison is not controlled, however: the initialiser only touches `nn.Linear` modules, and once tying is enabled the output projection *is* the token embedding, so the same uniform initialisation also overwrites the embeddings, leaving them roughly 172 times smaller than the positional ones. That row varies two things at once and is not evidence against tying itself.

Part 1.B is far stronger (Table 4): rank 32 with $\alpha = 64$ at 5×10^{-4} reaches 22.94 development and 20.87 test perplexity while training 1.77M parameters, 1.4% of GPT2’s 126M. Rank 32 beats rank 16, and the lower learning rate is worst. The final comparison (Table 5, Figure 5) gives a 42.73% test perplexity reduction while training 11.7 times fewer parameters, so both mandatory requirements are met.

The mechanism is worth stating precisely: pretrained GPT2 alone scores 137.09 here, *worse* than the scratch model, since Penn Treebank is heavily preprocessed and unlike GPT2’s training text; one epoch of adaptation drops it to 27.96. The gain therefore comes from adapting transferable representations, not from pretraining alone. One caveat: Part 1.A batches padded sentences while Part 1.B packs blocks crossing sentence boundaries, so part of the improvement reflects that extra context rather than LoRA alone.

4. References

- [1] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *International Conference on Learning Representations*, 2019. [Online]. Available: <https://openreview.net/forum?id=Bkg6RiCqY7>
- [2] O. Press and L. Wolf, “Using the output embedding to improve language models,” in *Proceedings of EACL*, 2017, pp. 157–163. [Online]. Available: <https://aclanthology.org/E17-2025/>
- [3] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” in *International Conference on Learning Representations*, 2022. [Online]. Available: <https://openreview.net/forum?id=nZcVKeFYf9>

Table 1: *Dataset statistics. Part 1.A uses sentence-level batches; Part 1.B packs the tokenized corpus into fixed-length blocks.*

Split	Sentences	Part 1.B blocks	Purpose
Train	42,068	19,729	Parameter learning
Development	3,370	1,561	Model selection
Test	3,761	1,768	Final held-out evaluation
Tokenizer vocabulary: 50,257 GPT2 BPE tokens; EOS/PAD id 50,256			
Part 1.A block size 128 tokens; Part 1.B block size 64 tokens			

Table 2: *Part 1.A learning-rate search, architecture fixed at the baseline. Bold marks the selected rate.*

Experiment	LR	Best epoch	Dev PPL	Test PPL
base_lr1e-3	0.0010	5	44.91	40.36
base_lr5e-4	0.0005	5	47.58	42.37
base_lr3e-4	0.0003	5	52.83	46.80
base_lr1e-4	0.0001	5	80.66	70.67

Table 3: *Part 1.A incremental experiments. Each row adds one change and keeps it. Bold marks the configuration selected on development perplexity.*

Experiment	LR	d_{model}	Heads	Layers	FF	Drop	Tied	Dev PPL	Test PPL
step0_baseline	0.001	128	2	2	512	0.0	No	44.91	40.36
step1_width192	0.001	192	2	2	512	0.0	No	43.25	38.82
step2_heads4	0.001	192	4	2	512	0.0	No	43.16	38.75
step3_depth3	0.001	192	4	3	512	0.0	No	41.46	37.44
step4_ffn768	0.001	192	4	3	768	0.0	No	41.30	37.46
step5_dropout01	0.001	192	4	3	768	0.1	No	40.49	36.44
step6_tying	0.001	192	4	3	768	0.1	Yes	53.68	47.45

Parameters: 13.28M (step0), 20.02M (step1–2), 20.36M (step3), 20.66M (step4–5), 11.01M (step6, tied).

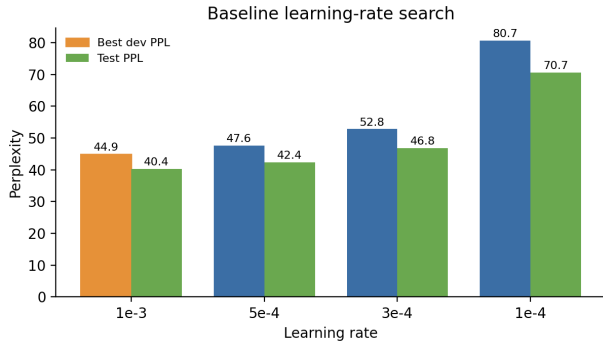
Table 4: *Part 1.B LoRA experiments. The GPT2 backbone is frozen and only the adapters are trained. Bold marks the selected model.*

Experiment	Rank	α	LR	Drop	Best ep.	Dev PPL	Test PPL
lora_r16_a32_lr5e4	16	32	0.0005	0.05	5	23.72	21.58
lora_r32_a64_lr5e4	32	64	0.0005	0.05	5	22.94	20.87
lora_r32_a64_lr3e4	32	64	0.0003	0.05	5	23.82	21.64

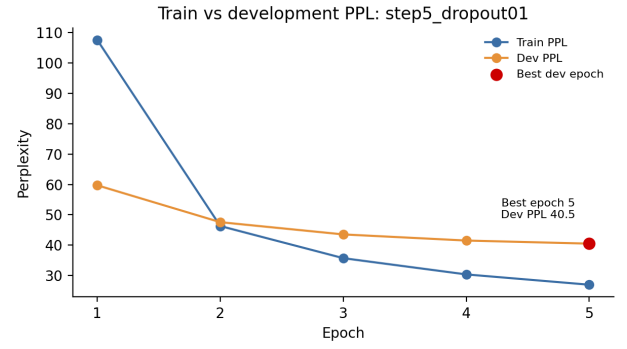
Trainable parameters: 0.88M at rank 16 and 1.77M at rank 32, of 126.21M total.

Table 5: *Final comparison between the best scratch model and the best LoRA-adapted GPT2. Bold marks the stronger system.*

Part	Selected configuration	Dev PPL	Test PPL	Trained params
Part 1.A	step5_dropout01	40.49	36.44	20.66M
Part 1.B	lora_r32_a64_lr5e4	22.94	20.87	1.77M
Relative reduction	—	43.34%	42.73%	11.7x fewer

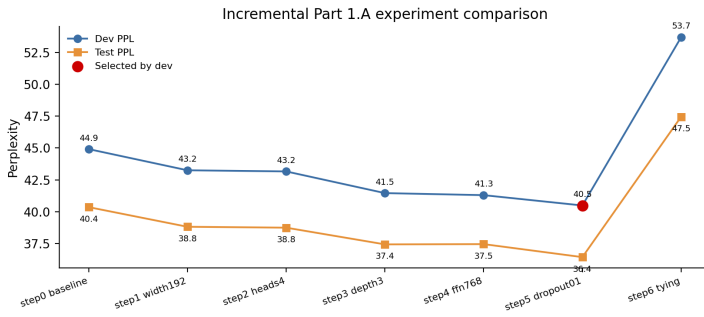


(a) Learning-rate search; the baseline is best at 10^{-3} .

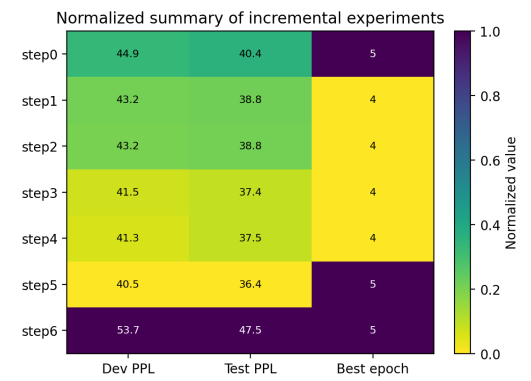


(b) Train and development perplexity of the selected model.

Figure 1: Part 1.A learning-rate selection and training dynamics.



(a) Development and test perplexity across the incremental changes.



(b) Normalised summary of the same experiments.

Figure 2: Part 1.A architecture comparison. The final row is weight tying, whose regression is explained by the initialisation interaction described in Section 3.

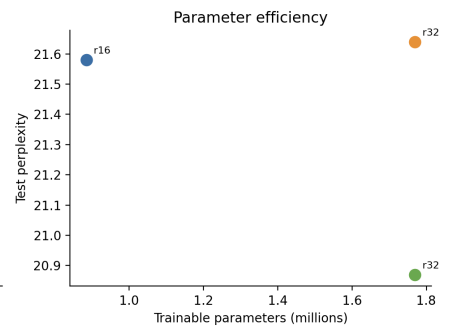
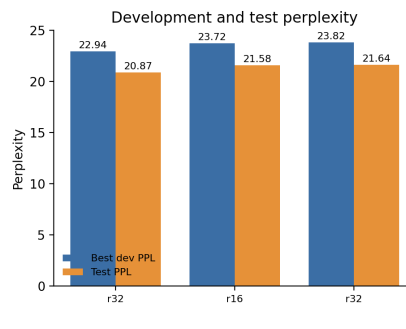
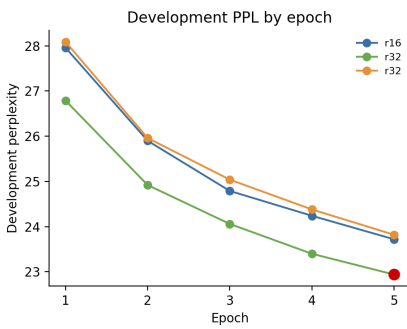


Figure 3: Part 1.B LoRA results. Left: development perplexity per epoch for the three configurations. Centre: best development and test perplexity per configuration. Right: trainable parameters against test perplexity.

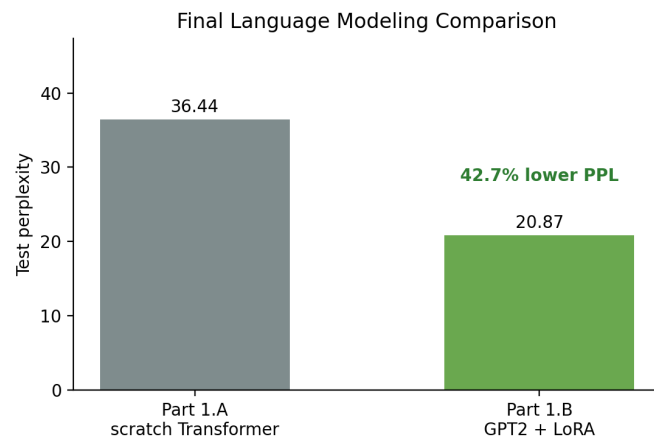


Figure 4: *Final test perplexity: the Part 1.A scratch model against the Part 1.B LoRA-adapted GPT2.*